

The following additional libraries are needed to run this notebook. Note that running on Colab is experimental, please report a Github issue if you have any problem.

```
!pip install d2l==1.0.3
```

Word Similarity and Analogy

:label: `sec_synonyms`

In [:numref:sec_word2vec_pretraining](#), we trained a word2vec model on a small dataset, and applied it to find semantically similar words for an input word. In practice, word vectors that are pretrained on large corpora can be applied to downstream natural language processing tasks, which will be covered later in [:numref:chap_nlp_app](#). To demonstrate semantics of pretrained word vectors from large corpora in a straightforward way, let's apply them in the word similarity and analogy tasks.

```
import os
import torch
from torch import nn
from d2l import torch as d2l
```

Loading Pretrained Word Vectors

Below lists pretrained GloVe embeddings of dimension 50, 100, and 300, which can be downloaded from the [GloVe website](#). The pretrained fastText embeddings are available in multiple languages. Here we consider one English version (300-dimensional "wiki.en") that can be downloaded from the [fastText website](#).

```
##@save
d2l.DATA_HUB['glove.6b.50d'] = (d2l.DATA_URL + 'glove.6B.50d.zip',
                                '0b8703943ccdb6eb788e6f091b8948e82231b6c41')

##@save
d2l.DATA_HUB['glove.6b.100d'] = (d2l.DATA_URL + 'glove.6B.100d.zip',
                                 'cd43bfb07e44e6f27cbcc7bc9ae380284fda45a')

##@save
d2l.DATA_HUB['glove.42b.300d'] = (d2l.DATA_URL + 'glove.42B.300d.zip',
                                  'b5116e234e9eb9076672cfeabf5469f3eec904fa')

##@save
d2l.DATA_HUB['wiki.en'] = (d2l.DATA_URL + 'wiki.en.zip',
                           'c1816da3821ae9f43899be655002f6c723e91b88')
```

"@save" is not an allowed annotation - allowed values include [[@param](#), [@title](#), [@markdown](#)].

@save" is not an allowed nnotation - allowed values 1clude [[@param](#), [@title](#), [@markdown](#)].

"@save" is not an allowed annotation - allowed values include [[@param](#), [@title](#), [@markdown](#)].

@save" is not an allowed nnotation - allowed values 1clude [[@param](#), [@title](#), [@markdown](#)].

[Hide code](#)

To load these pretrained GloVe and fastText embeddings, we define the following `TokenEmbedding` class.

```
##@save
class TokenEmbedding:
    """Token Embedding."""
    def __init__(self, embedding_name):
        self.idx_to_token, self.idx_to_vec = self._load_embedding(embedding_name)
        self.unknown_idx = 0
        self.token_to_idx = {token: idx for idx, token in enumerate(self.idx_to_token)}

    def _load_embedding(self, embedding_name):
        idx_to_token, idx_to_vec = ['<unk>'], []
        data_dir = d2l.download_extract(embedding_name)
        # GloVe website: https://nlp.stanford.edu/projects
        # fastText website: https://fasttext.cc/
        with open(os.path.join(data_dir, 'vec.txt'), 'r') as f:
            for line in f:
                elems = line.rstrip().split(' ')
                token, elems = elems[0], [float(elem) for elem in elems[1:]]
                # Skip header information, such as the top
                if len(elems) > 1:
                    idx_to_token.append(token)
                    idx_to_vec.append(elems)
        idx_to_vec = [[0] * len(idx_to_vec[0])] + idx_to_vec
        return idx_to_token, torch.tensor(idx_to_vec)
```

"@save" is not an allowed annotation - allowed values include [[@param](#), [@title](#), [@markdown](#)].

[Hide code](#)

```
def __getitem__(self, tokens):
    indices = [self.token_to_idx.get(token, self.unknown_token_idx)
                for token in tokens]
    vecs = self.idx_to_vec[torch.tensor(indices)]
    return vecs

def __len__(self):
    return len(self.idx_to_token)
```

Below we load the 50-dimensional GloVe embeddings (pretrained on a Wikipedia subset). When creating the `TokenEmbedding` instance, the specified embedding file has to be downloaded if it was not yet.

```
glove_6b50d = TokenEmbedding('glove.6b.50d')
```

```
Downloading ../data/glove.6B.50d.zip from http://d21-data.s3-accelerate.amazonaws.com/glove.6B.50d.zip...
```

Output the vocabulary size. The vocabulary contains 400000 words (tokens) and a special unknown token.

```
len(glove_6b50d)
```

```
400001
```

We can get the index of a word in the vocabulary, and vice versa.

```
glove_6b50d.token_to_idx['beautiful'], glove_6b50d.idx_to_token[3367]
```

```
(3367, 'beautiful')
```

✓ Applying Pretrained Word Vectors

Using the loaded GloVe vectors, we will demonstrate their semantics by applying them in the following word similarity and analogy tasks.

Word Similarity

Similar to `subsec_apply-word-embed`, in order to find semantically similar words for an input word based on cosine similarities between word vectors, we implement the following `knn` (k -nearest neighbors) function.

```
def knn(W, x, k):
    # Add 1e-9 for numerical stability
    cos = torch.mv(W, x.reshape(-1,)) / (
        torch.sqrt(torch.sum(W * W, axis=1) + 1e-9) *
        torch.sqrt((x * x).sum()))
    _, topk = torch.topk(cos, k=k)
    return topk, [cos[int(i)] for i in topk]
```

Then, we search for similar words using the pretrained word vectors from the `TokenEmbedding` instance `embed`.

```
def get_similar_tokens(query_token, k, embed):
    topk, cos = knn(embed.idx_to_vec, embed[[query_token]], k + 1)
    for i, c in zip(topk[1:], cos[1:]): # Exclude the input word
        print(f'cosine sim={float(c):.3f}: {embed.idx_to_token[int(i)]}')
```

The vocabulary of the pretrained word vectors in `glove_6b50d` contains 400000 words and a special unknown token. Excluding the input word and unknown token, among this vocabulary let's find three most semantically similar words to word "chip".

```
get_similar_tokens('chip', 3, glove_6b50d)
```

```
cosine sim=0.856: chips
cosine sim=0.749: intel
cosine sim=0.749: electronics
```

Below outputs similar words to "baby" and "beautiful".

```
get_similar_tokens('baby', 3, glove_6b50d)
```

```
cosine sim=0.839: babies
cosine sim=0.800: boy
cosine sim=0.792: girl
```

```
get_similar_tokens('beautiful', 3, glove_6b50d)
```

```
cosine sim=0.921: lovely  
cosine sim=0.893: gorgeous  
cosine sim=0.830: wonderful
```

Word Analogy

Besides finding similar words, we can also apply word vectors to word analogy tasks. For example, “man” : “woman” :: “son” : “daughter” is the form of a word analogy: “man” is to “woman” as “son” is to “daughter”. Specifically, the word analogy completion task can be defined as: for a word analogy $a : b :: c : d$, given the first three words a , b and c , find d . Denote the vector of word w by $\text{vec}(w)$. To complete the analogy, we will find the word whose vector is most similar to the result of $\text{vec}(c) + \text{vec}(b) - \text{vec}(a)$.

```
def get_analogy(token_a, token_b, token_c, embed):  
    vecs = embed[[token_a, token_b, token_c]]  
    x = vecs[1] - vecs[0] + vecs[2]  
    topk, cos = knn(embed.idx_to_vec, x, 1)  
    return embed.idx_to_token[int(topk[0])] # Remove unknown words
```

Let's verify the "male-female" analogy using the loaded word vectors.

```
get_analogy('man', 'woman', 'son', glove_6b50d)
```

```
'daughter'
```

Below completes a “capital-country” analogy: “beijing” : “china” :: “tokyo” : “japan”. This demonstrates semantics in the pretrained word vectors.

```
get_analogy('beijing', 'china', 'tokyo', glove_6b50d)
```

```
'japan'
```

For the “adjective-superlative adjective” analogy such as “bad” : “worst” :: “big” : “biggest”, we can see that the pretrained word vectors may capture the syntactic information.

```
get_analogy('bad', 'worst', 'big', glove_6b50d)
```

```
'biggest'
```

To show the captured notion of past tense in the pretrained word vectors, we can test the syntax using the “present tense-past tense” analogy: “do” : “did” :: “go” : “went”.

```
get_analogy('do', 'did', 'go', glove_6b50d)
```

```
'went'
```

Summary

- In practice, word vectors that are pretrained on large corpora can be applied to downstream natural language processing tasks.
- Pretrained word vectors can be applied to the word similarity and analogy tasks.

Exercises

1. Test the fastText results using `TokenEmbedding('wiki.en')`.
2. When the vocabulary is extremely large, how can we find similar words or complete a word analogy faster?

[Discussions](#)

